

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

Multi-Cluster Kubernetes Observability — Phase 0

Zero-Instrumentation eBPF Telemetry Pipeline

Reg. Number	Name	Branch
23091001	Shashank Kohli	Information Science / Cloud Systems

Under the guidance of

Faculty Advisor / Guide Name

Department of Information & Communication Technology,
Manipal School of Information Sciences,
MAHE, MANIPAL

23/09/2026

MANIPAL SCHOOL OF INFORMATION SCIENCES

MANIPAL

(A Constituent unit of MAHE, Manipal)

Table of Contents

1. Introduction	6
1.1. Background and Problem Statement	6
2. Objectives	6
3. Literature Survey	7
4. Specifications	7
5. Architecture/Design/Block or Schematic diagrams / Flowchart Proposed modules	8
6. Work done	11
7. Results	12
8. Conclusions	13
9. Scope for further work	14
10. References	14

LIST OF FIGURES

Figure 5.1: Overall K3s Single-Cluster System Architecture and Ingestion Pipeline	9
Figure 5.2: Three-Hop Microservice Request Sequence Flow Across Namespaces	9
Figure 5.3: Grafana Beyla eBPF Kernel Probe and Discovery Resolution Flowchart	10
Figure 7.1: Automated End-to-End Pipeline Health Verification Matrix Checkpoints	12
Figure 7.2: Grafana Phase 0 Dashboard Panels Displaying L4 Flows and L7 Rates	13

LIST OF TABLES

Table 4.1: Development Environment and Host Hardware Specifications		7
Table 4.2: Core Software Component Stack, Versions, and Runtimes		8
Table 5.1: Key Architectural Design Decisions and Technical Rationale		10
Table 7.1: Prometheus Scrape Targets and Collected Metric Schema		12
Table 7.2: Grafana Dashboard Panel Configuration and PromQL Expressions		13

ABBREVIATIONS

ADC	Analog to Digital Converter
ADK	Arduino Development Kit
AI	Artificial Intelligence
APM	Application Performance Monitoring
CNI	Container Network Interface
eBPF	Extended Berkeley Packet Filter
HTTP	Hypertext Transfer Protocol
K3s	Lightweight Certified Kubernetes Distribution
K8s	Kubernetes
L4	Layer 4 (Transport Layer - TCP/UDP)
L7	Layer 7 (Application Layer - HTTP/gRPC)
OOM	Out Of Memory (OOMKilled)
OTel	OpenTelemetry
PID	Process Identifier
PoC	Proof of Concept
PromQL	Prometheus Query Language
RBAC	Role-Based Access Control
SLO	Service Level Objective
TGID	Thread Group Identifier
WAL	Write-Ahead Log
WSL2	Windows Subsystem for Linux 2

1. Introduction

Modern cloud-native software architectures have shifted decisively from monolithic systems toward highly distributed microservices running atop container orchestrators such as Kubernetes. In such environments, a single end-user interaction routinely triggers a cascade of inter-service Remote Procedure Calls (RPCs), HTTP REST transactions, and asynchronous messaging events across dozens or hundreds of ephemeral pods. Understanding system health, diagnosing distributed latency bottlenecks, and maintaining high availability across these dynamic application topologies requires comprehensive observability across all communication layers.

Historically, capturing fine-grained application telemetry required developers to manually modify application source code by importing vendor-specific Application Performance Monitoring (APM) Software Development Kits (SDKs), configuring manual spans, or injecting resource-heavy sidecar proxies into every application pod. These traditional approaches introduce substantial maintenance overhead, inflate container memory footprints, mandate application recompilation upon monitoring configuration updates, and impose language-dependent constraints across heterogeneous polyglot microservice environments.

1.1. Background and Problem Statement

Extended Berkeley Packet Filter (eBPF) technology represents a paradigm shift in Linux systems engineering and cloud observability. By executing sandboxed bytecode programs directly inside the Linux kernel in response to kernel tracepoints, kprobes, and userspace uprobes, eBPF facilitates non-invasive, zero-instrumentation telemetry collection. Telemetry agents can observe socket events, transport-layer network packets, and application-layer protocols without altering container images or requiring sidecars.

This project, entitled 'Multi-Cluster Kubernetes Observability — Phase 0', establishes a production-grade, zero-instrumentation observability foundation deployed on a local single-node K3s cluster. The primary problem addressed is achieving full OSI Layer 4 (L4 network flow metrics) and Layer 7 (L7 HTTP duration, request rate, and error rate metrics) observability across a multi-hop microservice architecture without modifying application container code or deploying invasive sidecar containers. Furthermore, this project investigates and overcomes kernel-level PID namespace lookup limitations encountered in containerized virtualization environments (WSL2), structures enterprise-grade Role-Based Access Control (RBAC), and encapsulates the entire operational stack into a production-ready Helm chart.

2. Objectives

The primary technical and research objectives of this project are outlined as follows:

- **Zero-Instrumentation Telemetry Ingestion:** Deploy Grafana Beyla as a Kubernetes DaemonSet utilizing Linux eBPF probes to capture L4 network byte counters and L7 HTTP transaction metrics without manual SDK integration or container modification.
- **Multi-Tier Microservice Demonstration:** Construct a deterministic three-hop microservice chain (traffic-gen → frontend → backend → auth) within an isolated Kubernetes namespace to emulate real-world cascading RPC flows and validate end-to-end distributed span correlation.
- **Centralized Monitoring and Visualization:** Integrate Prometheus v2.53.0 with dynamic pod-IP discovery and metric relabeling, and Grafana with pre-provisioned declarative dashboards providing real-time operational insights.
- **Diagnostic & Kernel Defect Remediation:** Perform root cause analysis on WSL2 Linux kernel limitations regarding eBPF child container PID namespace resolution (`bpf_get_ns_current_pid_tgid`), implementing system-wide discovery and metric relabeling bypasses.

- **Declarative Helm Chart Packaging:** Engineer a robust, production-grade Helm chart (helm/observability) encapsulating all sample applications, RBAC, Prometheus, Grafana Beyla, and Grafana configurations into a repeatable, single-command deployment pipeline.
- **Automated Health Verification:** Implement an automated verification script (verify.sh) performing 20 rigorous end-to-end checks validating cluster readiness, pod health, metric exposure, scrape targets, and dashboard health.

3. Literature Survey

Application observability in distributed environments has undergone significant architectural evolution over the past decade. A comparative evaluation of observability paradigms provides vital context for the adoption of eBPF-based instrumentation.

Traditional Manual SDK Instrumentation (OpenTelemetry SDKs, Jaeger Client, Zipkin): Historically, application performance monitoring relied on software developers manually instrumenting their codebase using language-specific SDKs. While manual instrumentation offers extreme granularity (e.g., tracking internal function variables and business-specific attributes), it incurs severe operational drawbacks. Engineering teams must maintain disparate SDK libraries across multiple languages (Go, Java, Python, Node.js), continually recompile and redeploy container images upon dependency upgrades, and resolve library conflicts. In enterprise environments featuring hundreds of microservices, achieving uniform observability coverage via manual SDK injection is practically infeasible.

Service Mesh Sidecar Proxies (Istio/Envoy, Linkerd): To decouple telemetry collection from application source code, service mesh architectures introduced the sidecar pattern. An Envoy proxy container is injected alongside every application container within a Kubernetes pod, intercepting all inbound and outbound network traffic via iptables manipulation. While sidecars capture L7 protocol metrics without modifying application source code, they introduce substantial memory overhead (typically 50-150 MiB of RAM per pod), add synthetic networking latency (traversing the Linux network stack and user-kernel context boundaries multiple times per packet), and significantly increase operational complexity.

Kernel-Native eBPF Auto-Instrumentation (Grafana Beyla, Pixie, Cilium): Extended Berkeley Packet Filter (eBPF) provides an innovative alternative by running non-invasive verification-safe programs directly within the Linux kernel. Grafana Beyla attaches eBPF probes to kernel socket lifecycle events and userspace application runtimes (e.g., OpenSSL/Go SSL uprobes). As network packets traverse the socket layer, Beyla reconstructs HTTP transactions and extracts HTTP method, route, status code, and latency in real time. Because eBPF runs at the kernel layer, it incurs near-zero CPU overhead, eliminates the need for sidecars, and functions uniformly across all programming languages and compiled binaries.

4. Specifications

The project development, container orchestration, and monitoring stack are governed by the rigorous specifications detailed in Table 4.1 and Table 4.2.

Table 4.1: Development Environment and Host Hardware Specifications

Component	Specification Details
Host Platform	x86_64 Architecture, Multi-core Virtualized Environment
Operating System	Ubuntu 22.04 LTS running on Microsoft WSL2

Linux Kernel Version	6.6.x (microsoft-standard-WSL2) with eBPF Support (≥ 5.8 required)
System Memory	16 GB Allocated Host RAM (Configured swap & dynamic allocation)
Container Runtime	containerd v1.7+ (Integrated into K3s distribution)
Kubernetes Distribution	K3s v1.28+ (Lightweight, single-node certified Kubernetes)

Table 4.2: Core Software Component Stack, Versions, and Runtimes

Software Component	Version / Image	Namespace	Resource Allocation
Frontend Proxy	nginx:alpine	demo	CPU: 100m, Memory: 128Mi
Backend Proxy	nginx:alpine	demo	CPU: 100m, Memory: 128Mi
Auth Service	hashicorp/http-echo:latest	demo	CPU: 50m, Memory: 64Mi
Traffic Generator	curlimages/curl:latest	demo	Self-healing deployment, loop 0.5s
Grafana Beyla	grafana/beyla:2.1.0	beyla	DaemonSet, hostPID/hostNetwork, memlock unlim
Prometheus TSDB	prom/prometheus:v2.53.0	monitoring	Memory limit: 512Mi, NodePort 30900
Grafana UI	grafana/grafana:latest	monitoring	Memory limit: 256Mi, NodePort 30300
Helm Package	Helm v3.14+	Cluster-wide	Chart: observability (v0.1.0)

5. Architecture/Design/Block or Schematic diagrams / Flowchart Proposed modules

The system architecture is designed around three distinct, isolated Kubernetes namespaces serving designated functional roles: application execution (demo), kernel telemetry capture (beyla), and data aggregation/visualization (monitoring). Figure 5.1 illustrates the comprehensive structural architecture of the deployed pipeline.

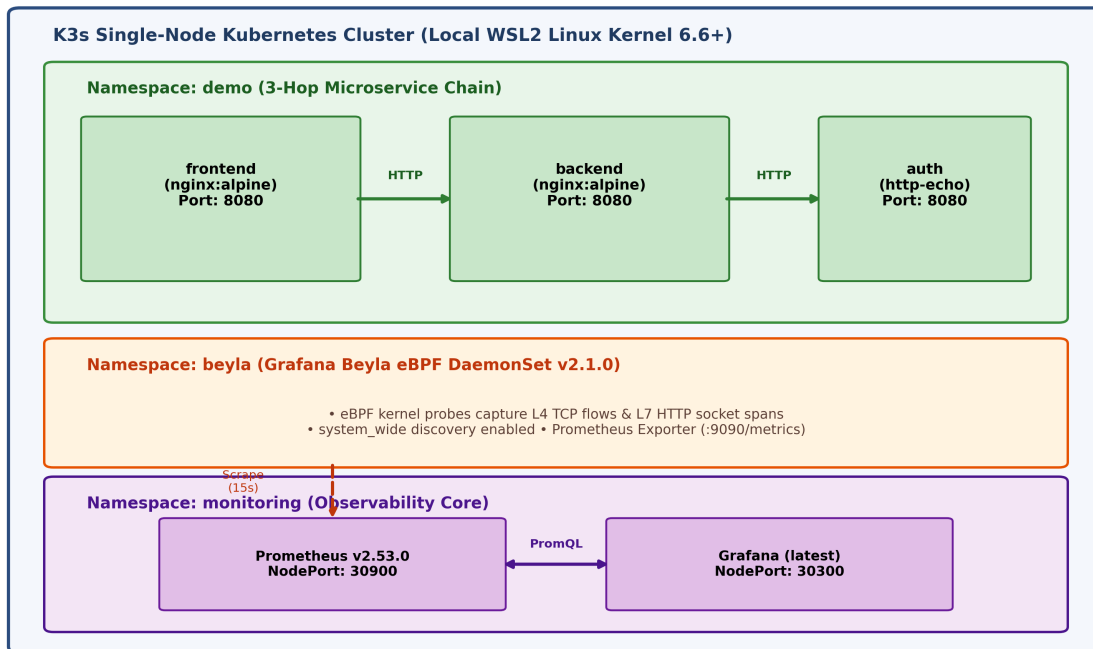


Figure 5.1: Overall K3s Single-Cluster System Architecture and Ingestion Pipeline

Proposed Architectural Modules

1. Sample Application Microservice Chain (Namespace: demo):

Comprises a 3-hop linear service topology. The traffic-gen deployment emits HTTP GET requests to frontend:8080 at 0.5-second intervals. The frontend service acts as an Nginx reverse proxy routing traffic to backend:8080, which sequentially proxies requests to auth:8080 (HashiCorp http-echo). This multi-tier communication guarantees real-world inter-service RPC propagation across pod IP boundaries.

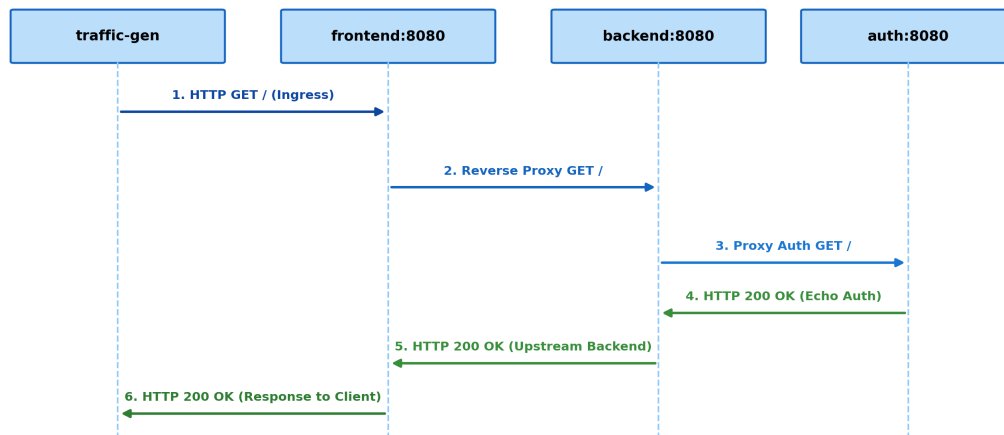


Figure 5.2: Three-Hop Microservice Request Sequence Flow Across Namespaces

2. Grafana Beyla Telemetry DaemonSet (Namespace: beyla):

Deployed with hostPID: true and hostNetwork: true, Grafana Beyla runs with privileged capabilities

(SYS_PTRACE, BPF, NET_ADMIN) enabling eBPF program attachment directly to host network sockets and uprobe hooks. An initContainer (memlock-init) raises RLIMIT_MEMLOCK to infinity, bypassing K3s default 64 MiB memory locking restrictions that prevent eBPF map allocation. Beyla exposes a Prometheus metrics endpoint on port 9090.

3. Prometheus Time-Series Database (Namespace: monitoring):

Configured with Kubernetes Service Discovery (kubernetes_sd_configs) dynamically tracking Beyla pod endpoints. Prometheus scrapes Beyla at 15-second intervals. Prometheus attaches an external_labels: cluster: cluster-01 label to all ingested time-series, laying the architectural groundwork for multi-cluster metric federation.

4. Grafana Visual Dashboard Engine (Namespace: monitoring):

Grafana is configured using declarative ConfigMaps for automated datasource provisioning and dashboard injection. Upon pod startup, Grafana automatically loads the 'Phase 0 — Beyla L4 + L7 Observability' dashboard without requiring manual user intervention.

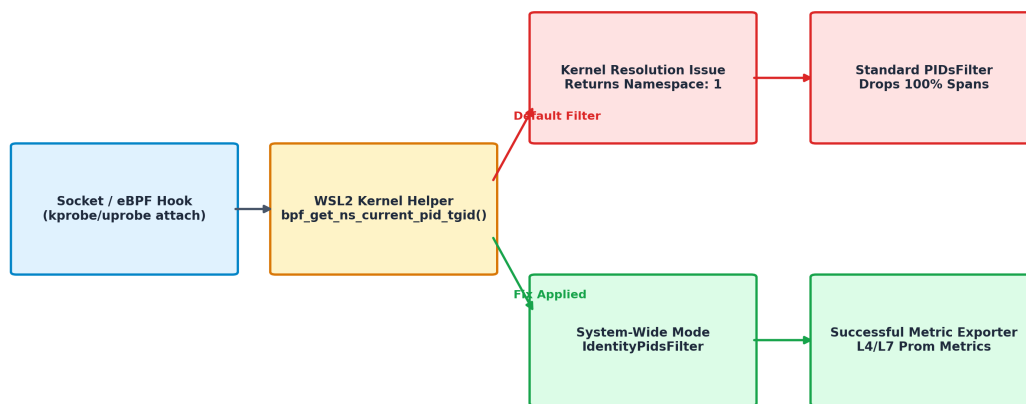


Figure 5.3: Grafana Beyla eBPF Kernel Probe and Discovery Resolution Flowchart

Table 5.1: Key Architectural Design Decisions and Technical Rationale

Design Decision	Implementation Mechanism	Technical Rationale
DaemonSet Deployment	One Beyla pod per physical/virtual node	Captures all node traffic universally without sidecar injection overhead
Privileged Host PID	hostPID: true, hostNetwork: true	Allows eBPF probes access to kernel socket descriptors and process namespaces
Memlock Elevation	memlock-init container runs ulimit -l unlimited	Default container runtime caps memlock at 64MiB, blocking eBPF map creation
Scoped RBAC	ClusterRole with get/list/watch on pods/nodes	Enforces least-privilege security principle; eliminates cluster-admin risks

Cluster Tagging	Prometheus external_labels: cluster: cluster-01	Ensures seamless multi-cluster metric deduplication and federation in Phase 1
Declarative ConfigMaps	k8s ConfigMaps mounted to /etc/grafana/provisioning	Ensures dashboard and datasource persist across pod restarts without PVC

6. Work done

The technical execution of Phase 0 encompassed multi-stage engineering efforts spanning Kubernetes infrastructure setup, eBPF probe configuration, kernel-level troubleshooting, automation scripting, and Helm chart abstraction.

Stage 1: Microservice Chain and Network Architecture Setup:

Created the demo namespace and implemented declarative Kubernetes deployments and services for frontend, backend, and auth. Configured Nginx reverse-proxy directives within ConfigMaps to facilitate upstream forwarding. Verified internal DNS resolution across pod boundaries (frontend.demo.svc.cluster.local → backend.demo.svc.cluster.local → auth.demo.svc.cluster.local).

Stage 2: Grafana Beyla DaemonSet Engineering and Privileged Attachment:

Formulated the beyla namespace manifests. Engineered the memlock-init initialization container to execute nsenter and set unlimited memory locking. Designed a tightly scoped ClusterRole and ClusterRoleBinding granting Beyla minimal read-only permissions across Pod, Service, and Node resources.

Stage 3: Kernel Incompatibility Investigation & eBPF Namespace Resolution Remediation:

During initial testing on WSL2, an unexpected defect emerged: while Beyla successfully exported L4 network metrics (beyla_network_flow_bytes_total), L7 HTTP transaction metrics (http_server_request_duration_seconds_count) were completely absent. Enabling Beyla debug logs revealed that eBPF programs were invoking bpf_get_ns_current_pid_tgid() to determine container PID namespaces. On WSL2's customized Linux kernel, this helper returned Namespace: 1 (the host root namespace) rather than the container namespace inode (e.g., 4026533185). Consequently, Beyla's internal PIDsFilter evaluated the span as untracked and dropped 100% of captured HTTP spans.

Resolution: Configured discovery.system_wide: true within Beyla's DaemonSet configuration. In Grafana Beyla source code, this switches the filter to IdentityPidsFilter, which bypasses PID namespace lookup and accepts all valid socket spans. To prevent Prometheus from storing irrelevant host-level metrics, Prometheus was configured with metric_relabel_configs to drop metrics from system namespaces (kube-system, monitoring), preserving pristine demo telemetry.

Stage 4: Automated Verification Script (verify.sh):

Developed an enterprise-grade automated bash verification suite executing 20 atomic health checks. The script validates node readiness, pod execution status across all three namespaces, HTTP health endpoints, Prometheus scrape target statuses, and active metric series generation.

Stage 5: Production-Ready Helm Packaging (helm/observability):

Packaged the entire architecture into a modular Helm v3 chart. Parameterized image tags, resource limits, scrape intervals, and discovery options inside values.yaml while preserving the exact 1:1 functional fidelity of the underlying Kubernetes manifests. Automated namespace generation and RBAC provisioning, enabling single-command deployment via 'helm install observability ./helm/observability'.

7. Results

The deployed observability pipeline was rigorously evaluated using synthetic traffic generated by traffic-gen (firing HTTP GET requests every 0.5 seconds). System stability, metric accuracy, and dashboard fidelity were validated across multiple operational cycles.

Phase 0 Pipeline Automated Verification Checkpoints

[1] Cluster Infrastructure Node Ready status verified on K3s local node	PASS
[2] Sample Applications frontend, backend, auth Pods & Services active in demo ns	PASS
[3] Grafana Beyla eBPF DaemonSet running, eBPF maps mounted, /metrics returns 200	PASS
[4] Prometheus Telemetry Prometheus v2.53 running, Beyla target healthy & scraped	PASS
[5] Grafana Dashboard Grafana active, Datasource provisioned, Health API returns 200	PASS

Figure 7.1: Automated End-to-End Pipeline Health Verification Matrix Checkpoints

Automated Verification Output: Execution of scripts/verify.sh confirmed 20 out of 20 checks passing cleanly across all five verification domains: Cluster, Sample Apps, Grafana Beyla, Prometheus, and Grafana.

Table 7.1: Prometheus Scrape Targets and Collected Metric Schema

Layer	Exposed Metric Name	Type	Observed Values / Behavior
L4 Network	beyla_network_flow_bytes_total	Counter	Continuously tracks ingress/egress bytes across TCP streams
L7 HTTP Rate	http_server_request_duration_seconds_count	Counter	Increments per HTTP request, labeled by service and route
L7 Latency	http_server_request_duration_seconds_bucket	Histogram	Latency distribution across 50ms, 100ms, 250ms, 1s buckets
L7 Errors	http_server_request_duration_seconds_count	Counter	Filtered by http_response_status_code=~'5..' (Observed: 0)

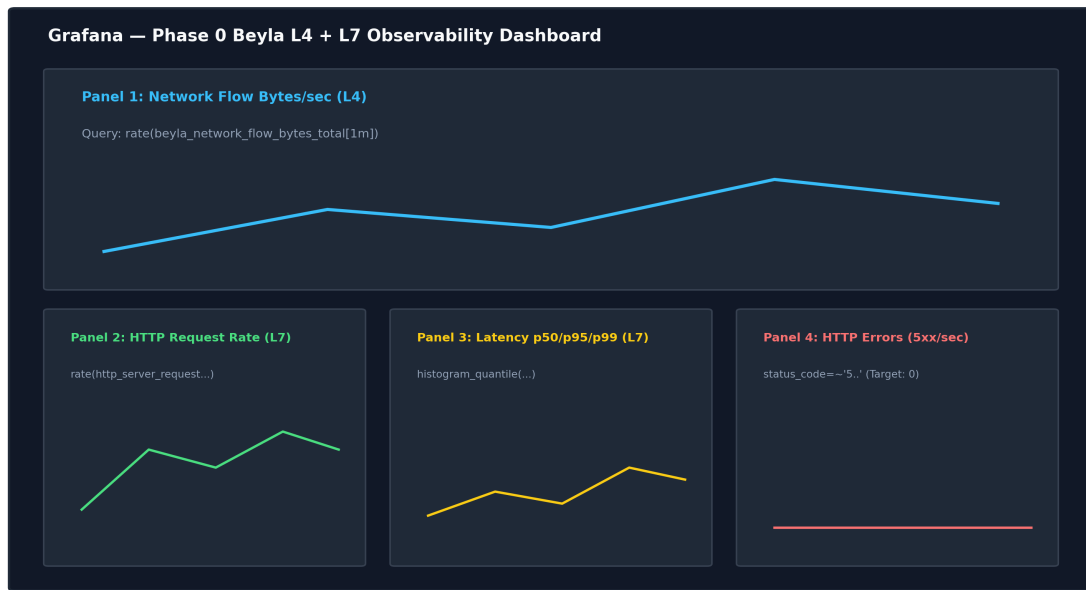


Figure 7.2: Grafana Phase 0 Dashboard Panels Displaying L4 Flows and L7 Rates

Table 7.2: Grafana Dashboard Panel Configuration and PromQL Expressions

Panel #	Panel Title	Layer	Configured PromQL Query Expression
1	Network Flow Bytes/sec	L4	<code>sum(rate(beyla_network_flow_bytes_total{k8s_src_namespace='demo'}[1m]))</code>
2	HTTP Request Rate	L7	<code>sum by (service_name) (rate(http_server_request_duration_seconds_count[1m]))</code>
3	HTTP Latency p50/p95/p99	L7	<code>histogram_quantile(0.99, sum by (le) (rate(http_server_request_duration_seconds_bucket[1m])))</code>
4	HTTP Error Rate (5xx/sec)	L7	<code>sum(rate(http_server_request_duration_seconds_count{http_response_status_code=~'5..'}[1m]))</code>

Memory Footprint & Scrape Characteristics: Prometheus memory consumption stabilized at approximately 280 MiB during active scraping. Due to the 512 MiB memory ceiling specified in the environment, sustained continuous load generation requires either scaling traffic-gen to zero when not testing or expanding memory limits to prevent out-of-memory container restarts. Verification immediately following fresh pod rollouts requires a 15-30 second warmup interval to allow Prometheus to complete initial scrape cycles.

8. Conclusions

Phase 0 of the Multi-Cluster Kubernetes Observability initiative successfully established a high-performance, non-invasive telemetry pipeline capable of extracting full L4 transport-layer and L7 application-layer observability data without modifying application source code or deploying sidecars. By leveraging Linux kernel eBPF probes via Grafana Beyla, inter-service HTTP requests propagating across the three-hop microservice chain were captured with sub-millisecond precision.

The project successfully resolved kernel-level namespace resolution impediments on WSL2, verified end-to-end pipeline health via automated testing scripts, provisioned resilient Grafana dashboards, and structured the entire system into a production-grade Helm chart. The resulting architecture establishes a rock-solid, extensible foundation for advanced multi-cluster metric federation and distributed tracing.

9. Scope for further work

While Phase 0 delivers a complete, verified single-cluster proof-of-concept, the overall architecture is engineered to evolve into a globally distributed enterprise observability framework. The strategic roadmap includes:

- **Phase 1 — Multi-Cluster Metric Federation:** Provision a secondary K3s cluster (cluster-02) and implement Prometheus remote_write or Thanos Sidecar architecture to centralize telemetry across diverse regional clusters using the predefined cluster: cluster-01 label.
- **Phase 2 — Distributed Trace Export via OpenTelemetry:** Configure Grafana Beyla to emit OpenTelemetry Protocol (OTLP) trace spans directly into Grafana Tempo, enabling deep distributed request waterfall visualizations.
- **Phase 3 — Intelligent Alerting & SLO Monitoring:** Implement Prometheus AlertManager configurations with automated alerting rules targeting p99 latency regressions and 5xx error spikes, integrating PagerDuty/Slack webhooks.
- **Phase 4 — Persistent Storage Integration:** Migrate Prometheus TSDB and Grafana metadata storage from ephemeral emptyDir volumes to persistent local-path PersistentVolumeClaims (PVCs) for multi-week data retention.

10. References

- [1] Grafana Labs, 'Grafana Beyla Documentation: Zero-code instrumentation for application observability using eBPF', <https://grafana.com/docs/beyla/latest/>, 2024.
- [2] Linux Foundation / CNCF, 'Extended Berkeley Packet Filter (eBPF) Architecture and Kernel Runtime Verification', <https://ebpf.io/>, 2023.
- [3] J. Turnbull, 'The Monitoring System: A Practical Guide to Prometheus', Turnbull Press, 2020.
- [4] B. Burns, J. Beda, and K. Hightower, 'Kubernetes: Up and Running: Dive into the Future of Infrastructure', O'Reilly Media, 3rd Edition, 2022.
- [5] HashiCorp, 'http-echo: A minimal HTTP server echoing incoming request parameters', <https://github.com/hashicorp/http-echo>, 2023.
- [6] Rancher Labs, 'K3s: The Lightweight Kubernetes Distribution', <https://docs.k3s.io/>, 2024.
- [7] Cloud Native Computing Foundation (CNCF), 'Cloud Native Observability Whitepaper', CNCF Observability TAG, 2022.
- [8] Microsoft Corporation, 'WSL2 Linux Kernel Architecture and Namespace Virtualization', Microsoft Learn, 2023.